ICPC Algorithm Library

队伍

目录

1 Basic

1.1 test

```
export fs="-fsanitize=address,undefined"
ulimit -s 1024000
ulimit -v 1024000
mk() { g++ -o $1 $1.cpp -O2; }
```

1.2 pai

```
pai() {
  mk a; mk bf; mk gen
  while true; do
    ./gen > 1.in
    ./bf < 1.in > 1.ans
    ./a < 1.in > 1.out
    if diff -Zq 1.out 1.ans; then echo ac; else echo wa; break; fi
  done
}
```

1.3 run

```
shopt -s nullglob # 没有匹配到任何文件时，让它返回空
ulimit -s 1024000
g++ $1.cpp -o $1 -std=c++20 -O2 -fsanitize=address,undefined || exit
for f in "$2"/*.in; do
  x=${f%.in}
  \time -f "$x: %es %MKB" ./$1 < $x.in > $1.out
  diff -Zq $1.out $x.ans
done
```

1.4 缺省源

```
#include <bits/stdc++.h>
using namespace std;

#define ALL(s) s.begin(), s.end()
#define SZ(s) int(s.size())
#define pb push_back

using LL = long long;
using ULL = unsigned long long;
using I = __int128;

void Cas() {
}
int main() {
  cin.tie(0)->sync_with_stdio(0);
  int t = 1;
  cin >> t;
  while (t--) {
    Cas();
  }
  return 0;
}
```

1.5 随机数

```
ULL seed = chrono::steady_clock::now().time_since_epoch().count();
mt19937_64 rnd(seed);
```

1.6 Matrix

```
template <class T>
struct Mat {
  int n, m;
  vector<T> a;
  Mat(int n = 0, int m = 0, T v = {}) : n(n), m(m), a(n * m, v) {}
  T& operator()(int i, int j) { return a[i * m + j]; }
};

using M = Mat<LL>;
M mul(M a, M b) {
  M c(a.n, b.m);
  for (int i = 0; i < a.n; i++) {
    for (int j = 0; j < b.m; j++) {
      for (int k = 0; k < a.m; k++) {
        c(i, j) = max(c(i, j), a(i, k) + b(k, j));
      }
    }
  }
  return c;
}
```

2 Graph

2.1 Dijkstra

```
struct Node {
  int u;
  LL d;
  bool operator<(const Node& o) const {
    return d > o.d;
  }
};
```

```
priority_queue<Node> q;
vector<LL> D(n, INF);
D[0] = 0;
q.push({0, 0});
while (SZ(q)) {
  auto [u, d] = q.top();
  q.pop();
  if (d != D[u]) continue;
  for (auto [v, w] : G[u]) {
    if (D[v] > d + w) {
      D[v] = d + w;
      q.push({v, D[v]});
    }
  }
}
```

2.2 Tarjan SCC

```
int tim = 0, scc = 0;
vector<int> dfn(n), low(n), bel(n, -1), stk;
auto tarj = [&](auto _, int u) -> void {
  dfn[u] = low[u] = ++tim;
  stk.push_back(u);
  for (int v : G[u]) {
    if (!dfn[v]) {
      _(_, v);
      low[u] = min(low[u], low[v]);
    } else if (bel[v] == -1) {
      low[u] = min(low[u], dfn[v]);
    }
  }
  if (dfn[u] != low[u]) return;
  while (1) {
    int v = stk.back();
    stk.pop_back();
    bel[v] = scc;
    if (v == u) break;
  }
  scc++;
};
for (int u = 0; u < n; u++) {
  if (!dfn[u]) tarj(tarj, u);
}
```

2.3 2-SAT

- $u_i = 2 \times u + i$。
- $u_a \vee v_b$：加入 $u_{a\oplus 1} \to v_b$ 和 $v_{b\oplus 1} \to u_a$。
- $u_a \Rightarrow v_b$：加入 $u_a \to v_b$ 和 $v_{b\oplus 1} \to u_{a\oplus 1}$。
- 强制 $u = a$：加入 $u_{a\oplus 1} \to u_a$。
- 禁止 $u = a$ 与 $v = b$ 同时成立：加入 $u_a \to v_{b\oplus 1}$ 和 $v_b \to u_{a\oplus 1}$。
- $u = v$：加入 $u_0 \leftrightarrow v_0$ 和 $u_1 \leftrightarrow v_1$。
- $u \neq v$：加入 $u_0 \leftrightarrow v_1$ 和 $u_1 \leftrightarrow v_0$。

其中每个 $\leftrightarrow$ 都表示正反两条有向边。

若存在 u 满足 u_0, u_1 在同一强连通分量中，则无解。

构造：$\text{ans}_u = [\text{bel}_{u_0} > \text{bel}_{u_1}]$。

2.4 Segment Tree Graph

- ZKW 线段树的叶子编号为 $S \sim 2S-1$，原图节点 i 对应叶子 $S+i-1$。
- sn 的值为 $2S-1$，表示一棵完整 ZKW 线段树的节点数。
- 原图节点编号为 $1 \sim n$。对于 $1 \le p \le sn$，id(p,0) 为 $n+p$，表示 down 树节点；id(p,1) 为 $n+sn+p$，表示 up 树节点。
- tot 初始为 $n+2(2S-1)$，表示两棵线段树建好后的最大节点编号；Node() 从 tot+1 开始追加辅助节点。

```
int S = 1;
while (S < n) S <<= 1;
int sn = 2 * S - 1;
int tot = n + 2 * sn;
vector<vector<pair<int, LL>>> G(tot + 1);
auto id = [&](int p, int up) {
  return n + p + up * sn;
};
auto add = [&](int u, int v, LL w) {
  G[u].push_back({v, w});
};
auto link = [&](int u, int v, LL w, int up) {
  if (up) swap(u, v);
  add(u, v, w);
};
auto Node = [&]() {
  G.emplace_back();
  return ++tot;
};
for (int up = 0; up < 2; up++) {
  for (int p = 1; p < S; p++) {
    link(id(p, up), id(p * 2, up), 0, up);
    link(id(p, up), id(p * 2 + 1, up), 0, up);
  }
  for (int i = 1; i <= n; i++) {
    link(id(S + i - 1, up), i, 0, up);
  }
}
auto mdf = [&](int l, int r, int x, LL w, int up) {
```

```
  for (l += S - 1, r += S; l < r; l >>= 1, r >>= 1) {
    if (l & 1) link(x, id(l++, up), w, up);
    if (r & 1) link(x, id(--r, up), w, up);
  }
};
```

```
// x -> [l, r]
mdf(l, r, x, w, 0);
// [l, r] -> x
mdf(l, r, x, w, 1);
// [l1, r1] -> [l2, r2]
int x = Node();
mdf(l1, r1, x, 0, 1);
mdf(l2, r2, x, w, 0);
```

3 Geometry

3.1 vector

```
using db = long double;
struct p2 {
  db x, y;
  db len() { return hypot(x, y); }
  p2 operator+(p2 b) { return {x + b.x, y + b.y}; }
  p2 operator-(p2 b) { return {x - b.x, y - b.y}; }
  p2 operator*(db k) { return {x * k, y * k}; }
  p2 operator/(db k) { return {x / k, y / k}; }
  db operator*(p2 b) { return x * b.x + y * b.y; }
  db operator%(p2 b) { return x * b.y - y * b.x; }
};
using ps = vector<p2>;
```

3.2 seg_poly

```
// pa 到 pb 的叉积，逆时针转为正
db side(p2 p, p2 a, p2 b) { return (a - p) % (b - p); }
db toL(p2 p, p2 a, p2 b) { return abs(side(p, a, b)) / (b - a).len(); }
```